OrangeMoon Design Document:

1. System Overview

Orange Moon uses a fixed sequential pipeline of four agents (A → B → C → D) to answer user questions about their menstrual cycle. The pipeline is orchestrated by a JavaScript controller in the frontend. Agents 1 and 2 are rule-based; Agents 3 and 4 each make an independent LLM call to Gemini 2.0 Flash. The pipeline runs reactively when the user asks a question, and proactively on app load to surface health warnings without user action.

2. Agent Roles, Inputs, and Outputs

Agent 1 — Data Collector

Role: Structures raw user data from localStorage into a JSON object for downstream agents. No AI involved.
Input: user question, cycle logs (flow, mood, symptoms, BBT), user settings (goal, cycle length, pregnancy mode).
Output: Structured JSON (period start dates, symptom frequency tally, last 7 days of logs, user goal and preferences)

Agent 2 — Cycle Analysis

Role: Performs all quantitative cycle reasoning. Deterministic logic only — no LLM.
Input: Structured JSON from Agent 1.
Output: Analysis report containing rolling average cycle length, current phase, next period prediction, ovulation date, fertile window, trend (stable/shortening/lengthening), and a list of detected irregularities with severity levels (low/medium/high).

Agent 3 — Health Flag Agent (Critic)

Role: Reviews Agent 2's irregularity list and reasons about clinical significance. Acts as a safety and validation layer before the Insight Agent responds.

Input: Irregularity list and cycle lengths from Agent 2.
Output: JSON array of flags, each with a severity level, a 1–2 sentence plain-language user message, and a showBanner boolean. Flags with medium or high severity are shown proactively as a banner on the Cycle screen. Falls back to rule-based flags if the LLM call fails.

Agent 4 — Insight Agent
Role: Final communicator. Receives all upstream outputs and generates a warm, concise (3–5 sentence) natural language response to the user's question. Does not re-analyze data — trusts Agents 2 and 3 entirely.
Input: User question, cycle analysis from Agent 2, health flags from Agent 3, user profile *from Agent 1.*
Output: Plain-language response displayed in the Chat UI. Conversation history (last 10 turns) is included for multi-turn coherence.

3. Coordination Mechanism

Pattern: Fixed sequential pipeline orchestrated by the frontend controller function runAgentPipeline(). Each agent receives the previous agent's output as input. Agents 1 and 2 run synchronously; Agents 3 and 4 run asynchronously. A loading indicator is shown while the pipeline executes. The pipeline cannot parallelize because each agent depends on the one before it.

4. Failure Handling

- Agent 3 LLM failure: Falls back to rule-based flags from Agent 2. Pipeline continues uninterrupted.
- Agent 4 LLM failure: Error message shown to user in the chat bubble. User can retry.

- No API key: Agents 3 and 4 stop immediately with a message directing the user to Settings. Rule-based cycle analysis still runs.
- Insufficient data (< 3 logs): Proactive health check is skipped. Predictions use the user's cycle length setting. Agent 4's system prompt instructs it to acknowledge limited data.

5. Tools and APIs

- Gemini 2.0 Flash (Google AI Studio) — LLM for Agents 3 and 4. Called via browser fetch with a user-provided API key.
- localStorage — on-device storage for all cycle logs and settings. Accessed by Agent 1.
- Vellum — used for visual workflow design and agent testing. The diagram reflects this pipeline directly.
- Orange Moon Frontend (HTML/CSS/JS) — single-file PWA hosting the pipeline controller, all four agent functions, and the Chat UI.